Tutorial Sheet: Queues & Trees

QUEUE

isEmpty(Q): returns true if the queue is empty, false otherwise.
delete(Q): deletes the element at the front of the queue and returns its value.
insert(Q, i): inserts the integer i at the rear of the queue.

1. Consider the following pseudocode. Assume that IntQueue is an integer queue. What does the function fun do?

```
void fun(int n)
{
    IntQueue q = new IntQueue();
    q.enqueue(0);
    q.enqueue(1);
    for (int i = 0; i < n; i++)
    {
        int a = q.dequeue();
        int b = q.dequeue();
        q.enqueue(b);
        q.enqueue(a + b);
        print(a);
    }
}
```

A. Prints numbers from 0 to n-1
B. Prints numbers from n-1 to 0
C. Prints first n Fibonacci numbers
D. Prints first n Fibonacci numbers in reverse order.

OPTION C, The function prints first n Fibonacci Numbers. Note that 0 and 1 are initially there in q. In every iteration of loop, sum of the two queue items is enqueued and the front item is dequeued.

Solution: https://www.techtud.com/example/example-queue

2. **What operation is performed by the function f?**

```
void f(queue Q)
{
    int i ;
    if (!isEmpty(Q))
    {
        i = delete(Q);
        f(Q);
        insert(Q, i);
    }
}
```

A. Leaves the queue Q unchanged
B. Reverses the order of the elements in the queue Q
C. Deletes the element at the front of the queue Q and inserts it at the rear keeping the other elements in the same order.
D. Empties the queue Q

OPTION B, 3. As it is recursive call, and we are removing from the front while inserting from end, that means last element will be deleted from the last and will be inserted 1st position in the new queue. And this will continue till the first call executes insert(Q,i) function. So, the queue will be in reverse

Solution: https://www.youtube.com/watch?v=_eti0VBB9ps&t=21s

3. An implementation of a queue Q, using two stacks S1 and S2, is given below. If n insert and m (<= n) delete operations were performed in an arbitrary order on an empty queue Q, x and y be the number of push and pop operations performed respectively in the process. Which one of the following is true for all values of m and n?

```
void insert(Q, x)
{
    push (S1, x);
}

void delete(Q)
{
    if(stack-empty(S2)) then
    {
        if(stack-empty(S1)) then
        {
            print("Q is empty");
            return;
        }
        else
        {
            while (!(stack-empty(S1)))
            {
                x = pop(S1);
                push(S2, x);
            }
            x = pop(S2);
        }
    }
}
```

A. **n+m <= x < 2n and 2m <= y <= n+m**
B. n+m <= x < 2n and 2m<= y <= 2n
C. 2m <= x < 2n and 2m <= y <= n+m
D. 2m <= x < 2n and 2m <= y <= 2n

https://www.youtube.com/watch?v=52C36j6K0Y0&t=38s

OPTION A, The order in which **insert** and **delete** operations are performed matters here.

The best case: Insert and delete operations are performed alternatively. In every delete operation, 2 pop and 1 push operations are performed. So, total m + n push (n push for insert() and m push for delete()) operations and 2m pop operations are performed.

The worst case: First n elements are inserted and then m elements are deleted. In first delete operation, n + 1 pop operations and n push operations are performed. Other than first, in all delete operations, 1 pop operation is performed. So, total m + n pop operations and 2n push operations are performed (n push for insert() and n push for delete())

4. Following is a C like pseudocode for a function that takes a Queue as an argument, and uses a stack S to do the processing. What does the function do in general?

```
void fun(Queue *Q)
{
    Stack S;    // Say it creates an empty stack S

    // Run while Q is not empty
    while (!isEmpty(Q))
    {
        // deQueue an item from Q and push the dequeued item to S
        push(&S, deQueue(Q));
    }

    // Run while Stack S is not empty
    while (!isEmpty(&S))
    {
        // Pop an item from S and enqueue the poppped item to Q
        enQueue(Q, pop(&S));
    }
}
```

A. Removes the last element from Q
B. Keeps the Q same as it was before the call
C. Makes Q empty
D. Reverses the Q

5. Consider the following operation along with Enqueue and Dequeue operations on a Queue, where k is a global parameter. What will be the worst case time complexity of a sequence of n MultiDequeue() operations if the queue is empty initially?

```
MultiDequeue(Q)
{
    m = k;
    while (Q is not empty and m  > 0)
    {
        Dequeue(Q)
        m = m - 1;
    }
}
```

A. O(n)	**B.** O(n + k)
C. O(nk)	**D.** O(n^2)

OPTION A, **O(n)**. Since the queue is empty initially, the condition of while loop will never become true. So the time complexity will be **O(n)**

6. **Suppose a Circular queue of capacity (n - 1) elements is implemented with an array of n elements. Assume that the insertion and deletion operation are carried out using REAR and FRONT as array index variables, respectively. Initially, REAR = FRONT = 0. The conditions to detect, queue is full and queue is empty will be:**

A. Full: (REAR+1) mod n == FRONT, Empty: REAR == FRONT
B. Full: (REAR+1) mod n == FRONT, Empty: (FRONT+1) mod n == REAR
C. Full: REAR == FRONT, Empty: (REAR+1) mod n == FRONT
D. Full: (FRONT+1) mod n == REAR, Empty: REAR == FRONT

OPTION A, Full: (REAR+1) mod n == FRONT, Empty: REAR == FRONT

https://www.youtube.com/watch?v=gYxdHbbjn_8

7. Which of the following is true about linked list implementation of queue?

A. In push operation, if new nodes are inserted at the beginning of linked list, then in pop operation, nodes must be removed from end.
B. In push operation, if new nodes are inserted at the end, then in pop operation, nodes must be removed from the beginning.
C. Both of the above
D. None of the above

OPTION C, Both of the above. To keep the **First In First Out** order, a queue can be implemented using linked list in any of the given two ways

Q.8

Let Q denote a queue containing sixteen numbers and S be an empty stack. Head(Q) returns the element at the head of the queue Q **without** removing it from Q. Similarly Top(S) returns the element at the top of S **without** removing it from S. Consider the algorithm given below.

```
while Q is not Empty do
    if S is Empty OR Top(S) ≤ Head(Q) then
        x := Dequeue(Q);
        Push(S,x);
    else
        x := Pop(S);
        Enqueue(Q,x);
    end
end
```

The maximum possible number of iterations of the while loop in the algorithm is_____ [This Question was originally a Fill-in-the-Blanks question]

A: 16

B: 32

C: 256

D: 64

Solution: **Answer: (C)**

Explanation: The worst case happens when the queue is sorted in decreasing order. In worst case, loop runs n*n times.

Q.9

Consider the following statements:

```
i.   First-in-first out types of computations are efficiently supported
by STACKS.

ii.  Implementing LISTS on linked lists is more efficient than
implementing LISTS on

     an array for almost all the basic LIST operations.
```

```
iii. Implementing QUEUES on a circular array is more efficient than
implementing QUEUES
     on a linear array with two indices.
iv.  Last-in-first-out type of computations are efficiently supported
by QUEUES.
```

Which of the following is correct?

A (ii) and (iii) are true

B (i) and (ii) are true

C (iii) and (iv) are true

D (ii) and (iv) are true

Answer: (A)

Explanation: i -STACK is the data structure that follows Last In First Out (LIFO) or First In Last Out (FILO) order, in which the element which is inserted at last is removed out first.
ii – Implementing LISTS on linked lists is more efficient than implementing it on an array for almost all the basic LIST operations because the insertion and deletion of elements can be done in O(1) in Linked List but it takes O(N) time in Arrays.

iii- Implementing QUEUES on a circular array is more efficient than implementing it on a linear array with two indices because using circular arrays, it takes less space and can reuse it again.

iv – QUEUE is the data structure that follows First In First Out (FIFO) or Last In Last Out (LILO) order, in which the element which is inserted first is removed first.

only (ii) and (iii) are TRUE.
Option (A) is correct.

Q.10

Consider a standard Circular Queue 'q' implementation (which has the same condition for Queue Full and Queue Empty) whose size is 11 and the elements of the queue are q[0], q[1], q[2].....,q[10]. The front and rear pointers are initialized to point at q[2] . In which position will the ninth element be added?

A q[0]

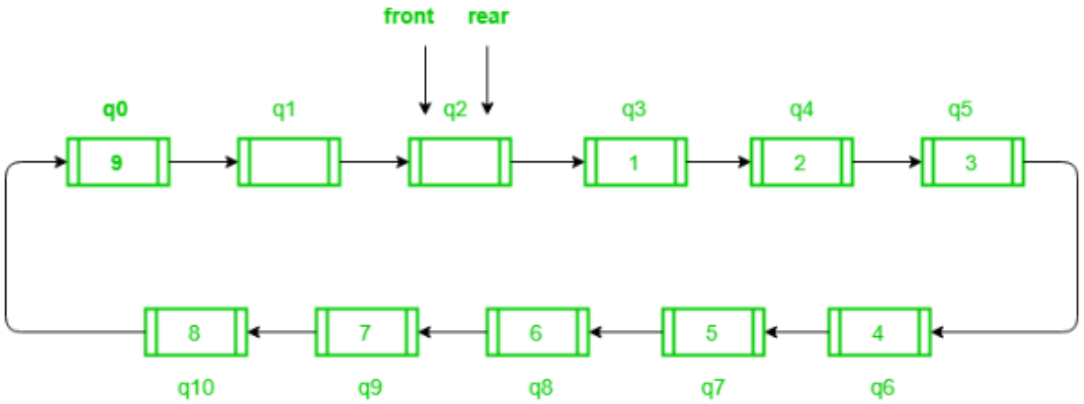
B q[1]

C q[9]

D q[10]

Answer: (A)

Explanation: Circular queue whose total size is 11, front and rear pointers are initialized to point at q[2]:



Therefore, 9th element will be added at pointer q[0].

So, option (A) is correct.

Q.11

Time Needed to Buy Tickets

There are `n` people in a line queuing to buy tickets, where the `0th` person is at the **front** of the line and the `(n - 1)th` person is at the **back** of the line.

You are given a **0-indexed** integer array `tickets` of length `n` where the number of tickets that the `ith` person would like to buy is `tickets[i]`.

Each person takes **exactly 1 second** to buy a ticket. A person can only buy **1 ticket at a time** and has to go back to **the end** of the line (which happens **instantaneously**) in order to buy more tickets. If a person does not have any tickets left to buy, the person will **leave** the line.

Return *the **time taken** for the person at position* `k` ***(0-indexed)*** *to finish buying tickets*.

Example 1:

```
Input: tickets = [2,3,2], k = 2

Output: 6

Explanation:

- In the first pass, everyone in the line buys a ticket and the line becomes [1, 2, 1].

- In the second pass, everyone in the line buys a ticket and the line becomes [0, 1, 0].

The person at position 2 has successfully bought 2 tickets and it took 3 + 3 = 6 seconds.
```

Example 2:

```
Input: tickets = [5,1,1,1], k = 0

Output: 8

Explanation:

- In the first pass, everyone in the line buys a ticket and the line becomes [4, 0, 0, 0].

- In the next 4 passes, only the person in position 0 is buying tickets.

The person at position 0 has successfully bought 5 tickets and it took 4 + 1 + 1 + 1 + 1 = 8 seconds.
```

Constraints:

- `n == tickets.length`
- `1 <= n <= 100`
- `1 <= tickets[i] <= 100`
- `0 <= k < n`

Solution:

https://www.youtube.com/watch?v=pJRJEZx1gzA

Number of Students Unable to Eat Lunch

The school cafeteria offers circular and square sandwiches at lunch break, referred to by numbers `0` and `1` respectively. All students stand in a queue. Each student either prefers square or circular sandwiches.

The number of sandwiches in the cafeteria is equal to the number of students. The sandwiches are placed in a **stack**. At each step:

- If the student at the front of the queue **prefers** the sandwich on the top of the stack, they will **take it** and leave the queue.
- Otherwise, they will **leave it** and go to the queue's end.

This continues until none of the queue students want to take the top sandwich and are thus unable to eat.

You are given two integer arrays `students` and `sandwiches` where `sandwiches[i]` is the type of the i^{th} sandwich in the stack (`i = 0` is the top of the stack) and `students[j]` is the preference of the j^{th} student in the initial queue (`j = 0` is the front of the queue). Return *the number of students that are unable to eat.*

Example 1:

```
Input: students = [1,1,0,0], sandwiches = [0,1,0,1]

Output: 0

Explanation:

- Front student leaves the top sandwich and returns to the end of the line making
students = [1,0,0,1].

- Front student leaves the top sandwich and returns to the end of the line making
students = [0,0,1,1].

- Front student takes the top sandwich and leaves the line making students =
[0,1,1] and sandwiches = [1,0,1].

- Front student leaves the top sandwich and returns to the end of the line making
students = [1,1,0].

- Front student takes the top sandwich and leaves the line making students = [1,0]
and sandwiches = [0,1].

- Front student leaves the top sandwich and returns to the end of the line making
students = [0,1].

- Front student takes the top sandwich and leaves the line making students = [1]
and sandwiches = [1].

- Front student takes the top sandwich and leaves the line making students = []
and sandwiches = [].
```

```
Hence all students are able to eat.
```

Example 2:

```
Input: students = [1,1,1,0,0,1], sandwiches = [1,0,0,0,1,1]

Output: 3
```

Solution:

https://www.youtube.com/watch?v=R4F4xwpBtZg